

DATA301 Assignment 4

Sina Zia, 300661655

Exercise 1

Sample quantile definitions

There is no unique way to calculate a sample quantile when the desired quantile falls between observed values. Wicklin (2017), following Hyndman and Fan (1996), compares nine commonly used definitions of sample quantiles. Types 1-3 use discrete rules based on rounding and order statistics, while Types 4-9 use continuous linear interpolation between ordered observations. As a result, the interpolation methods can return values between observations, whereas the discrete methods generally return an observed value or, in some cases, an average of adjacent observations.

Wicklin (2017) shows that the choice of definition matters most for small samples and when there are large gaps between adjacent observations. The differences between methods generally become much smaller as the sample size increases. This is relevant to this analysis because the later Monte Carlo experiment includes $n = 5$, where the choice of quantile definition may have a noticeable effect on the estimated first and third quartiles. Wicklin also shows that statistical software packages can use different default definitions, meaning the same data can produce different sample quantiles depending on the method used.

For the choice used in this analysis, I focus mainly on Types 6-9 because they have clear statistical interpretations that are directly relevant to selecting a quantile estimator. Types 1-3 use discontinuous rules, while Types 4 and 5 are valid interpolation methods but are not the methods emphasised by Hyndman and Fan in their final recommendation.

For this analysis, I use Type 8. Type 7 is the default method used by R, but Hyndman and Fan (1996) do not recommend it as their preferred general definition of a sample quantile. R retained Type 7 largely for consistency with earlier implementations, rather than because it was considered statistically superior. Type 8 is recommended by Hyndman and Fan because it produces approximately median-unbiased quantile estimates regardless of the underlying distribution. In repeated sampling, this means the estimated quantile is approximately equally likely to fall above or below the true population quantile. Type 6 is also a reasonable alternative because its probability positions are based on the expected positions of the order statistics, but Type 8 has the additional median-unbiased motivation and is the method recommended by Hyndman and Fan. Type 9 is less suitable here because its justification is specifically related to normally distributed data, whereas the data in this problem follow an exponential distribution.

R's `quantile()` function uses Type 7 by default. Therefore, Type 8 will need to be specified explicitly when the quartiles are calculated later in the analysis.

Bootstrap bias-corrected estimator

Let $\hat{\lambda}_{IQR}$ denote the IQR estimator calculated from the original sample. The bias of an estimator is

$$Bias(\hat{\lambda}_{IQR}) = E(\hat{\lambda}_{IQR}) - \lambda.$$

Since the true value of λ is unknown for an observed sample, the bias can be estimated using the bootstrap. Stanford University (2017) describes the nonparametric bootstrap as resampling observations from the original sample with replacement.

For very small samples, this can produce many repeated values. In this problem, this can result in $Q_1 = Q_3$, making the denominator of $\hat{\lambda}_{IQR}$ equal to zero. Because the observations are assumed to follow an exponential distribution, I instead use a parametric bootstrap. The original sample is first used to estimate λ using $\hat{\lambda}_{IQR}$, and each bootstrap sample is then generated using `rexp()` with rate equal to this estimate.

Let $\hat{\lambda}_{IQR,b}^*$ denote the IQR estimate calculated from the b -th bootstrap sample. Using $B = 200$ bootstrap replications, the bootstrap estimate of the bias is

$$\widehat{Bias}_{boot} = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR,b}^* - \hat{\lambda}_{IQR}.$$

The first-order bias-corrected estimator is

$$\tilde{\lambda} = \hat{\lambda}_{IQR} - \widehat{Bias}_{boot}.$$

Substituting the bootstrap estimate of bias gives

$$\tilde{\lambda} = \hat{\lambda}_{IQR} - \left(\frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR,b}^* - \hat{\lambda}_{IQR} \right),$$

and therefore

$$\tilde{\lambda} = 2\hat{\lambda}_{IQR} - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR,b}^*.$$

The IQR estimator is implemented using Type 8 quantiles, as selected above.

```
lambda_IQR <- function(x) {
  q <- quantile(x, probs = c(0.25, 0.75), type = 8)
  log(3) / (q[2] - q[1])
}
```

The first-order bootstrap bias-corrected estimator is implemented using 200 bootstrap replications.

```
lambda_tilde <- function(x, B = 200) {

  lambda_hat <- lambda_IQR(x)

  boot_estimates <- rep(0, B)

  for (b in 1:B) {
    x_boot <- rexp(n = length(x), rate = lambda_hat)
    boot_estimates[b] <- lambda_IQR(x_boot)
  }

  estimated_bias <- mean(boot_estimates) - lambda_hat

  lambda_corrected <- lambda_hat - estimated_bias

  return(lambda_corrected)
}
```

Monte Carlo comparison

A Monte Carlo experiment was used to compare the maximum likelihood estimator, the IQR estimator, and the bootstrap bias-corrected IQR estimator. For each combination of

$$\lambda \in \{0.1, 1, 10\}$$

and

$$n \in \{5, 30, 100\},$$

independent samples were generated from an exponential distribution with the specified value of λ . For each simulated sample, the three estimators were calculated and their squared errors were recorded. The bias-corrected estimator used $B = 200$ bootstrap replications.

The performance measure used to compare the estimators was mean squared error,

$$MSE(\hat{\lambda}) = E[(\hat{\lambda} - \lambda)^2].$$

In the simulation, this was estimated by averaging the squared errors across the Monte Carlo replications.

I used 10,000 Monte Carlo replications for $n = 5$, 3,000 for $n = 30$, and 1,500 for $n = 100$. More replications were allocated to the smallest sample size because estimates based on very small samples can vary substantially from one simulated sample to another. Within each sample size, I kept the same number of replications for all three values of λ so that the parameter settings were treated consistently.

To assess the numerical precision of the estimated MSEs for the chosen numbers of replications, I also calculated the Monte Carlo standard error of each estimated MSE,

$$MCSE = \frac{sd(Z)}{\sqrt{R}},$$

where

$$Z_r = (\hat{\lambda}_r - \lambda)^2$$

is the squared error from replication r , and R is the number of Monte Carlo replications. I also report the relative MCSE, $MCSE/MSE$, because it makes the numerical uncertainty easier to compare across settings where the MSEs differ greatly in scale. MCSE and relative MCSE are used only to assess the precision of the Monte Carlo estimates; MSE remains the sole performance measure used to compare the estimators, as required by the assignment.

```
lambda_ML <- function(x) {
  1 / mean(x)
}
```

```
monte_carlo <- function(lambda, n, R = 5000) {

  sq_error_ML <- rep(0, R)
  sq_error_IQR <- rep(0, R)
  sq_error_tilde <- rep(0, R)

  for (r in 1:R) {

    x <- rexp(n = n, rate = lambda)

    estimate_ML <- lambda_ML(x)
    estimate_IQR <- lambda_IQR(x)
    estimate_tilde <- lambda_tilde(x, B = 200)

    sq_error_ML[r] <- (estimate_ML - lambda)^2
    sq_error_IQR[r] <- (estimate_IQR - lambda)^2
```

```

    sq_error_tilde[r] <- (estimate_tilde - lambda)^2
  }

  data.frame(
    lambda = lambda,
    n = n,
    R = R,

    MSE_ML = mean(sq_error_ML),
    MCSE_ML = sd(sq_error_ML) / sqrt(R),
    Rel_MCSE_ML = (sd(sq_error_ML) / sqrt(R)) / mean(sq_error_ML),

    MSE_IQR = mean(sq_error_IQR),
    MCSE_IQR = sd(sq_error_IQR) / sqrt(R),
    Rel_MCSE_IQR = (sd(sq_error_IQR) / sqrt(R)) / mean(sq_error_IQR),

    MSE_tilde = mean(sq_error_tilde),
    MCSE_tilde = sd(sq_error_tilde) / sqrt(R),
    Rel_MCSE_tilde = (sd(sq_error_tilde) / sqrt(R)) / mean(sq_error_tilde)
  )
}

```

```

settings <- expand.grid(
  lambda = c(0.1, 1, 10),
  n = c(5, 30, 100)
)

```

```

results <- data.frame()

for (i in 1:nrow(settings)) {

  if (settings$n[i] == 5) {
    R_current <- 10000
  } else if (settings$n[i] == 30) {
    R_current <- 3000
  } else if (settings$n[i] == 100) {
    R_current <- 1500
  }

  current_result <- monte_carlo(
    lambda = settings$lambda[i],
    n = settings$n[i],
    R = R_current
  )
}

```

```

)

results <- rbind(results, current_result)
}

```

The estimated MSEs were:

lambda	n	R	ML	IQR	Bias-corrected IQR
0.1	5	10000	0.005705	0.009846	0.005406
1.0	5	10000	0.610869	1.028614	0.558739
10.0	5	10000	58.044737	88.841903	50.678742
0.1	30	3000	0.000382	0.000937	0.000817
1.0	30	3000	0.040763	0.096974	0.082261
10.0	30	3000	4.014249	9.139406	7.777777
0.1	100	1500	0.000107	0.000255	0.000243
1.0	100	1500	0.010511	0.022901	0.021928
10.0	100	1500	1.080179	2.371017	2.258291

The relative Monte Carlo standard errors were:

lambda	n	R	ML rel. MCSE (%)	IQR rel. MCSE (%)	Corrected rel. MCSE (%)
0.1	5	10000	4.13	5.52	5.24
1.0	5	10000	8.08	9.22	8.31
10.0	5	10000	4.76	5.29	5.27
0.1	30	3000	3.35	4.19	3.99
1.0	30	3000	3.15	4.07	3.76
10.0	30	3000	3.56	4.06	3.74
0.1	100	1500	4.06	4.72	4.52
1.0	100	1500	3.93	4.20	4.02
10.0	100	1500	4.01	4.23	4.10

The bootstrap bias correction reduced the MSE of the IQR estimator in all nine settings. The improvement was largest when $n = 5$, while the difference between the corrected and uncorrected IQR estimators became smaller as the sample size increased.

For $n = 30$ and $n = 100$, the maximum likelihood estimator had the lowest MSE for all three values of λ . For $n = 5$, the bias-corrected IQR estimator had the lowest estimated MSE in all three cases, although the Monte Carlo uncertainty was also greater for this small-sample setting. Overall, the simulation suggests that the bootstrap correction improves the performance of the IQR estimator in terms of MSE, while the maximum likelihood estimator has the lowest MSE once the sample size is moderately large.

Exercise 2

The `ausbeer` series is also discussed in Hyndman and Athanasopoulos (2018), which I consulted as a general reference when interpreting the plots. I treat this section as exploratory data analysis: I use plots together with simple summaries extracted directly from the series, without fitting a time-series model.

Initial time-series plot

```
library(fpp2)

data("ausbeer")

beer_df <- data.frame(
  time = as.numeric(time(ausbeer)),
  year = floor(as.numeric(time(ausbeer))),
  quarter = factor(
    cycle(ausbeer),
    levels = 1:4,
    labels = c("Q1", "Q2", "Q3", "Q4")
  ),
  production = as.numeric(ausbeer)
)

frequency(ausbeer)
```

```
## [1] 4
```

```
start(ausbeer)
```

```
## [1] 1956    1
```

```
end(ausbeer)
```

```
## [1] 2010    2
```

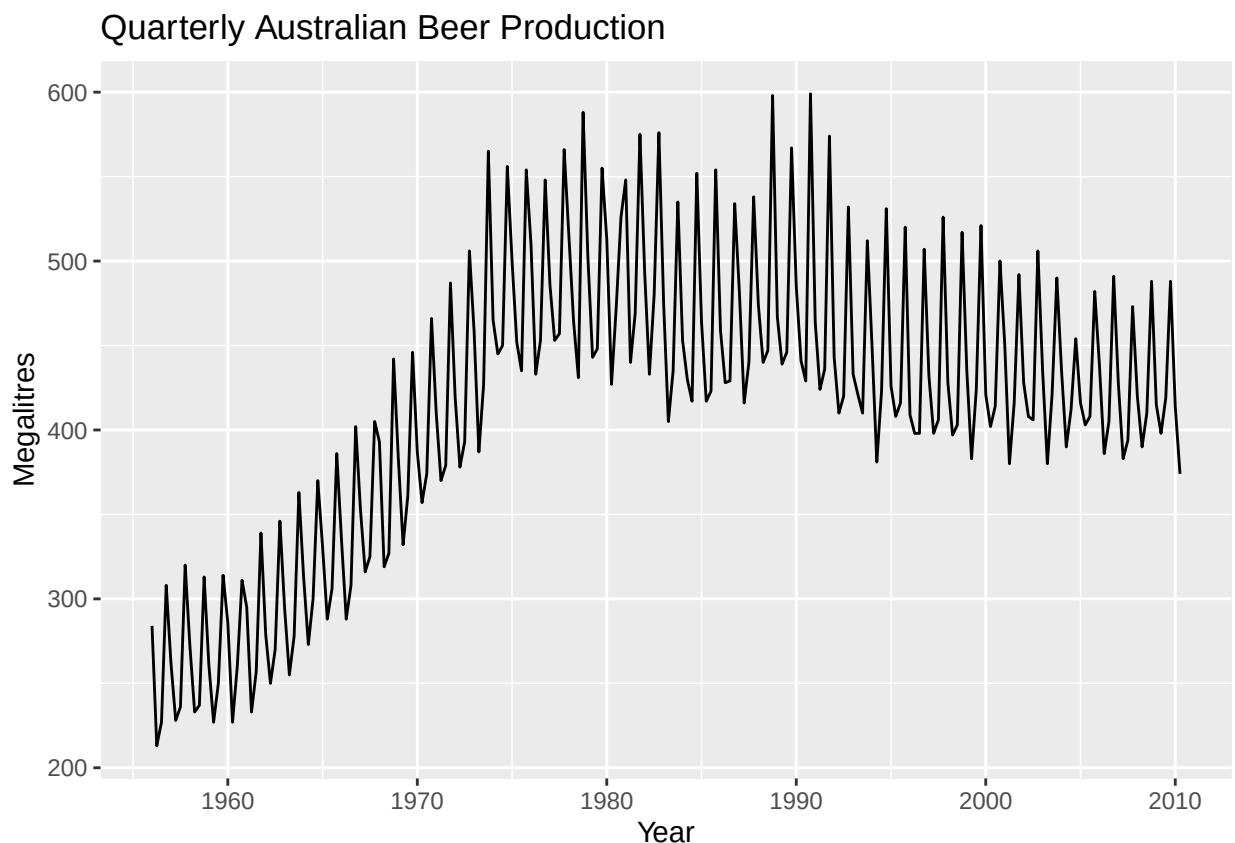
```
length(ausbeer)
```

```
## [1] 218
```

```
summary(ausbeer)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  213.0   379.2   422.0   415.4   465.8   599.0
```

```
autoplot(ausbeer) +
  labs(
    title = "Quarterly Australian Beer Production",
    x = "Year",
    y = "Megalitres"
  )
```



1. Frequency of the series

The frequency of the series is 4, as shown by `frequency(ausbeer)`. This means there are four observations per year, so the data are quarterly. The series begins in the first quarter of 1956 and ends in the second quarter of 2010, as shown by `start(ausbeer)` and `end(ausbeer)`. It contains 218 observations in total. The repeating within-year pattern visible in the time-series plot is also consistent with quarterly seasonality.

2. Overall behaviour and long-term trend

To make the long-term movement easier to inspect without fitting a model, I also calculated the mean production for each year and plotted those annual means.

```
# 2010 contains only Q1 and Q2, so exclude it when comparing full-year means.
complete_years <- beer_df[beer_df$year <= 2009, ]

annual_mean <- aggregate(production ~ year, data = complete_years, FUN = mean)
names(annual_mean)[2] <- "mean_production"

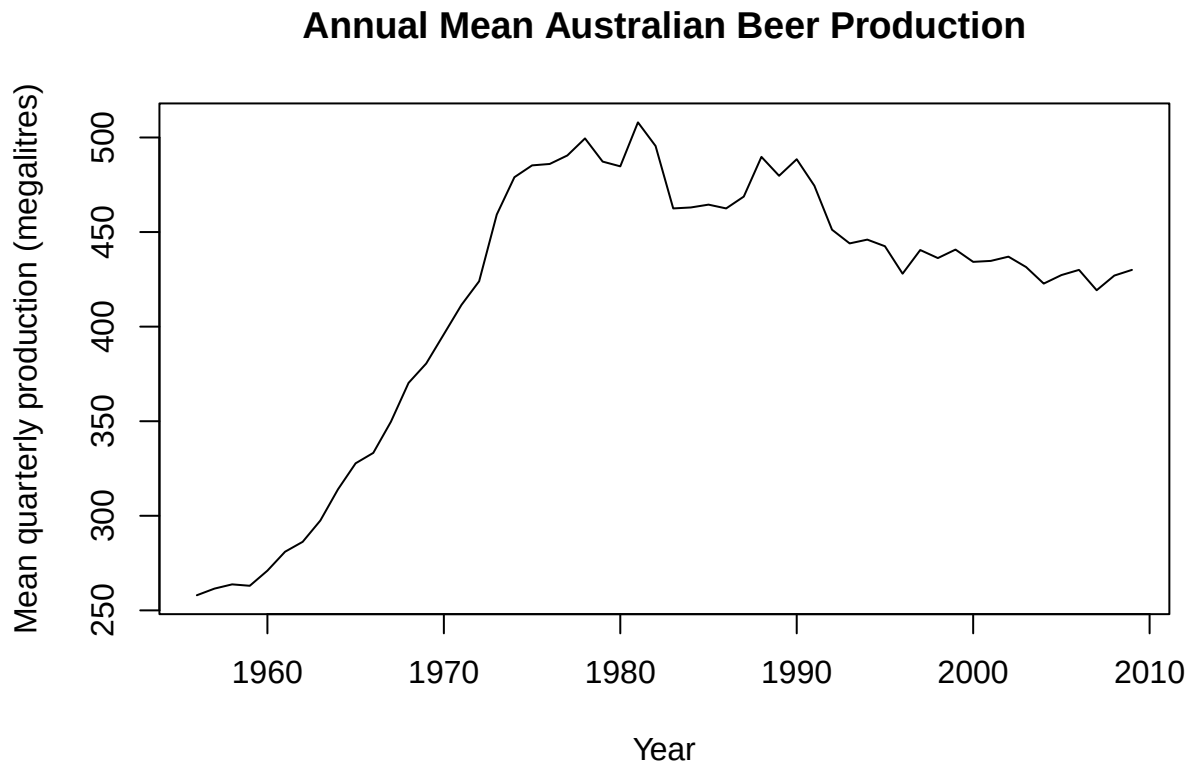
print(annual_mean[which.max(annual_mean$mean_production), ], row.names = FALSE)
```

```
## year mean_production
## 1981                508
```

```
print(annual_mean[which.min(annual_mean$mean_production), ], row.names = FALSE)
```

```
## year mean_production
## 1956                258
```

```
plot(
  annual_mean$year,
  annual_mean$mean_production,
  type = "l",
  main = "Annual Mean Australian Beer Production",
  xlab = "Year",
  ylab = "Mean quarterly production (megalitres)"
)
```



Beer production shows a clear change in long-term behaviour over the series. Production rises strongly from the late 1950s through the 1960s and early 1970s, then remains at a relatively high level through much of the late 1970s and 1980s. The highest full-year mean is 508 megalitres in 1981, compared with 258 megalitres in 1956. From around the early 1990s onward, the annual means generally decline. The annual-mean plot therefore supports the pattern seen in the quarterly series: there is a clear long-term movement, but it is not a single constant upward or downward trend across the whole period.

3. Seasonality

The following table gives the mean and median production for each quarter across the full series.

```
quarter_summary <- data.frame(  
  Quarter = levels(beer_df$quarter),  
  Mean = as.numeric(tapply(beer_df$production, beer_df$quarter, mean)),  
  Median = as.numeric(tapply(beer_df$production, beer_df$quarter, median))  
)  
  
knitr::kable(quarter_summary, digits = 1)
```

Quarter	Mean	Median
Q1	416.8	432.0
Q2	372.6	397.0
Q3	387.4	413.0
Q4	485.4	506.5

There is clear evidence of strong seasonality in the series. Q4 has the highest typical production, with a mean of 485.4 megalitres and a median of 506.5, while Q2 has the lowest, with a mean of 372.6 and a median of 397.0. Q1 is next highest, followed by Q3. This numerical ordering matches the repeating pattern visible in the time-series plot: production usually drops from Q1 to Q2, rises slightly in Q3, and then increases sharply in Q4. The seasonal pattern remains visible throughout the series, although the size of the seasonal fluctuations changes over time.

4. Variability and possible transformation

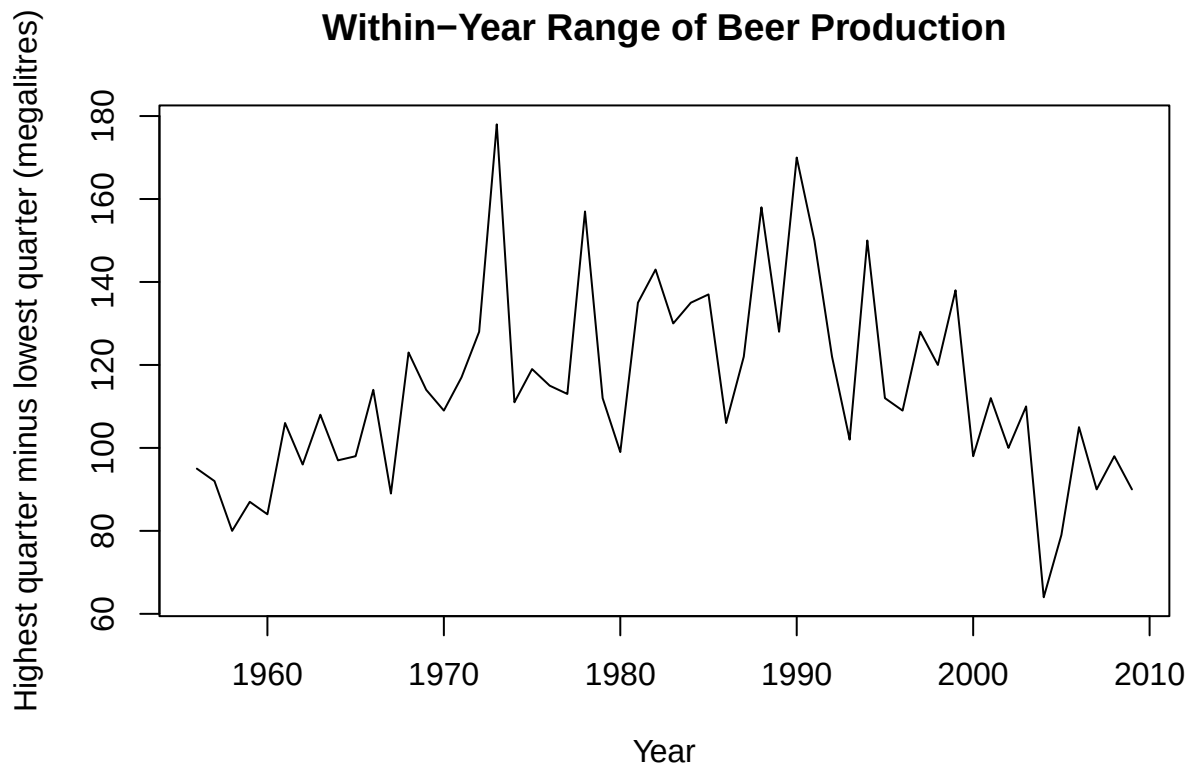
To examine whether the amount of within-year variation changes over time, I calculated the range between the highest and lowest quarterly production value for each complete year. The incomplete year 2010 is excluded from this calculation because it contains only Q1 and Q2.

```
annual_range <- aggregate(
  production ~ year,
  data = complete_years,
  FUN = function(x) max(x) - min(x)
)
names(annual_range)[2] <- "within_year_range"

summary(annual_range$within_year_range)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      64.0   98.0   112.0   114.5   128.0   178.0
```

```
plot(
  annual_range$year,
  annual_range$within_year_range,
  type = "l",
  main = "Within-Year Range of Beer Production",
  xlab = "Year",
  ylab = "Highest quarter minus lowest quarter (megalitres)"
)
```



The variability does not appear constant over time. Across complete years, the difference between the highest and lowest quarter ranges from 64 to 178 megalitres, with a median of 112. The within-year range is generally smaller in the early part of the series, becomes much larger in several years during the higher-production middle decades, and then falls again in parts of the 2000s. The relationship is not perfectly uniform from year to year, but the ordinary time-series plot and the range plot both suggest that the size of the seasonal fluctuations changes with the level of the series. I would therefore consider a variance-stabilising transformation, such as a log or Box-Cox transformation, before modelling rather than assuming constant variability.

5. Unusual observations

A high or low value is not automatically unusual in this series because the data are strongly seasonal. I therefore inspected both the most extreme production values and the largest year-to-year changes for the same quarter.

```
highest_values <- head(
  beer_df[order(-beer_df$production), c("year", "quarter", "production")],
  8
)

lowest_values <- head(
```

```

beer_df[order(beer_df$production), c("year", "quarter", "production")],
8
)

beer_df$year_on_year_change <- c(
  rep(NA, 4),
  diff(beer_df$production, lag = 4)
)

change_df <- beer_df[!is.na(beer_df$year_on_year_change), ]
largest_changes <- head(
  change_df[
    order(-abs(change_df$year_on_year_change)),
    c("year", "quarter", "production", "year_on_year_change")
  ],
  8
)

knitr::kable(
  highest_values,
  row.names = FALSE,
  caption = "Eight highest quarterly observations"
)

```

Table 4: Eight highest quarterly observations

year	quarter	production
1990	Q4	599
1988	Q4	598
1978	Q4	588
1982	Q4	576
1981	Q4	575
1991	Q4	574
1989	Q4	567
1977	Q4	566

```

knitr::kable(
  lowest_values,
  row.names = FALSE,
  caption = "Eight lowest quarterly observations"
)

```

Table 5: Eight lowest quarterly observations

year	quarter	production
1956	Q2	213
1956	Q3	227
1959	Q2	227
1960	Q2	227
1957	Q2	228
1958	Q2	233
1961	Q2	233
1957	Q3	236

```
knitr::kable(
  largest_changes,
  digits = 1,
  row.names = FALSE,
  caption = "Eight largest absolute year-to-year changes within the same quarter"
)
```

Table 6: Eight largest absolute year-to-year changes within the same quarter

year	quarter	production	year_on_year_change
1988	Q4	598	60
1973	Q4	565	59
1974	Q2	445	58
1982	Q1	493	-55
1981	Q4	575	49
1983	Q3	435	-45
1992	Q4	532	-42
1983	Q4	535	-41

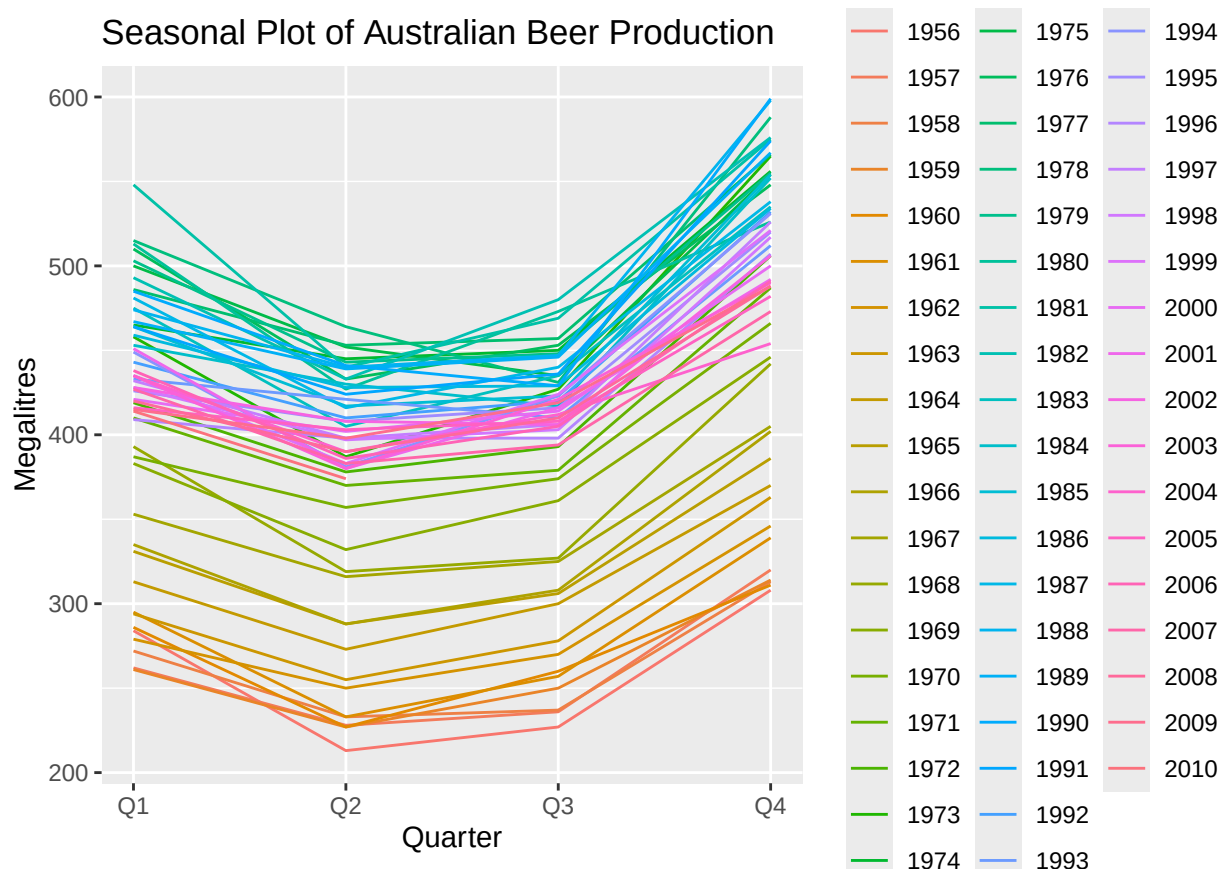
The extreme-value tables show why the seasonal context matters. All eight of the highest observations occur in Q4, which is normally the highest-production quarter, while most of the lowest observations occur in the early years and in Q2 or Q3. I would therefore not call an observation unusual simply because it is one of the overall highest or lowest values.

One of the clearest unusual candidates is 1988 Q4. Its production was 598 megalitres, the second-highest value in the entire series, and it increased by 60 megalitres from the same quarter in the previous year, the largest absolute year-to-year change in the table. The overall maximum, 599 megalitres in 1990 Q4, is also visually prominent, although being a Q4 value makes its high level less surprising. Another observation that attracts attention is 1982 Q1, which fell by 55 megalitres

from the previous Q1. I would describe these as exploratory candidates for unusual observations rather than formal outliers, because the series has both strong seasonality and a changing long-term level.

6. Seasonal plot

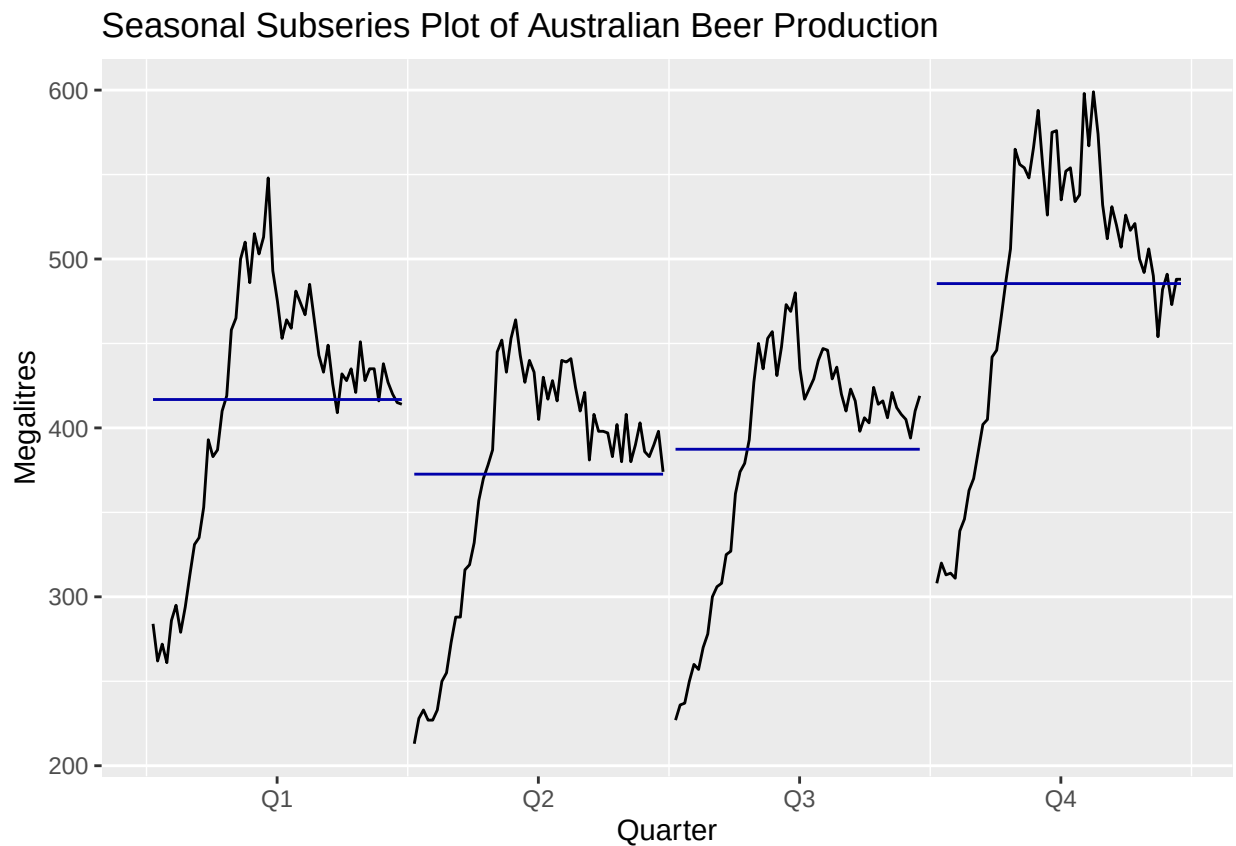
```
ggseasonplot(ausbeer) +  
  labs(  
    title = "Seasonal Plot of Australian Beer Production",  
    x = "Quarter",  
    y = "Megalitres"  
  )
```



The seasonal plot makes the within-year pattern much clearer than the ordinary time-series plot. Most years follow the same shape: production falls from Q1 to Q2, increases slightly in Q3, and then rises sharply in Q4. This agrees with the quarter summaries, where Q4 has the highest mean production and Q2 the lowest. Plotting each year across the four quarters also makes it easier to see that the basic seasonal shape is persistent even though the overall level of production differs substantially between years.

7. Seasonal subseries plot

```
ggsubseriesplot(ausbeer) +  
  labs(  
    title = "Seasonal Subseries Plot of Australian Beer Production",  
    x = "Quarter",  
    y = "Megalitres"  
  )
```



The seasonal subseries plot shows that Q4 has the highest average beer production, followed by Q1 and Q3, while Q2 has the lowest average production, as indicated by the blue horizontal lines.

The relative seasonal ordering is fairly stable over time, with Q4 typically having the highest values and Q2 generally having the lowest. However, the size of the differences between quarters changes over time, so the strength of the seasonal pattern is not completely constant.

The plot also shows that production within each quarter changes substantially over time rather than remaining at a constant level. When considered together with the ordinary time-series plot, these changes are consistent with the broader rise in production through the earlier part of the series and the later decline.

8. Detrending and transformation before modelling

Based on the exploratory evidence, I would detrend the series before modelling because its level changes systematically over time. The ordinary time-series plot, the annual-mean plot, and the seasonal subseries plot all show the early rise, the relatively high middle period, and the later decline. Removing that long-term movement would make it easier to model the remaining seasonal and short-term behaviour.

I would also consider a variance-stabilising transformation. The within-year range varies substantially across the series, from 64 to 178 megalitres, and the larger seasonal spreads occur mainly during periods when production is relatively high. This suggests that the variability is not completely independent of the level. A log or Box-Cox transformation could therefore be considered, although the exploratory plots alone do not establish which transformation would be best.

References

- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.). OTexts. <https://otexts.com/fpp2/>
- Hyndman, R. J., & Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician*, 50(4), 361–365. <https://doi.org/10.2307/2684934>
- Stanford University. (2017). *STATS 200: Introduction to Statistical Inference, Lecture 19: The Bootstrap*. <https://web.stanford.edu/class/archive/stats/stats200/stats200.1172/Lecture19.pdf>
- Wicklin, R. (2017). Sample quantiles: A comparison of 9 definitions. *SAS Blogs*. <https://blogs.sas.com/content/iml/2017/05/24/definitions-sample-quantiles.html>